Key Concepts

In-context Learning

In modern language models, tokens later in the context are easier to predict than tokens earlier in the context. As the context gets longer, loss goes down. In some sense this is just what a sequence model is designed to do (use earlier elements in the sequence to predict later ones), but as the ability to predict later tokens from earlier ones gets better, it can increasingly be used in interesting ways (such as specifying tasks, giving instructions, or asking the model to match a pattern) that suggest it can usefully be thought of as a phenomenon of its own. When thought of in this way, it is usually referred to as **in-context learning**.[3]

Emergent in-context learning was noted in GPT-2 [10] and gained significant attention in GPT-3 [1]. Simply by adjusting a "prompt", transformers can be adapted to do many useful things without re-training, such as translation, question-answering, arithmetic, and many other tasks. Using "prompt engineering" to leverage in-context learning became a popular topic of study and discussion [11, 12].

At least two importantly different ways of conceptualizing and measuring in-context learning exist in the literature. The first conception, represented in Brown *et al.*, focuses on *few-shot learning* [1] of specific tasks. The model is prompted with several instances of some "task" framed in a next-token-prediction format (such as few-digit addition, or English-to-French translation). The second conception of in-context learning, represented in Kaplan *et al.* [13], focuses on observing the loss at different *token indices*, in order to measure how much better the model gets at prediction as it receives more context. The first conception can be thought of as a micro perspective (focusing on specific tasks), where as the second conception can be seen as a macro perspective (focusing on general loss, which on average correlates with these tasks).

The "few-shot learning" conception of in-context learning has tended to receive greater community attention. The ability to do many different tasks with one large model, even without further fine-tuning, is a notable change to the basic economics of model training. Moreover, it gives evidence of wide-ranging general capabilities and the ability to adapt on the fly, which nudges us to re-examine what it means for a model to "understand" or to "reason".

However, for the purposes of this work, we focus instead on the Kaplan *et al*. conception: *decreasing loss* at *increasing token indices.* We do so because it's a more general framing of the phenomenon than "few-shot learning". A drawback of this definition is it fails to isolate specific behaviors of interest. At the same time, it allows us to measure models' overall ability to learn on-the-fly from the context, without depending on our specific choices of "task". We'll also see that, starting from this definition, we are *also* able to study a couple classic few-shot learning examples (see Argument 4).

Throughout this work we compute a simple heuristic measure of in-context learning:

- **In-context learning score:** the loss of the 500th token in the context minus the average loss of the 50th token in the context, averaged over dataset examples.

We chose the 500th and 50th token indices somewhat arbitrarily. The 500th token is near the end of a length-512 context, and the 50th token is far enough into the context that some basic properties of the text have been established (such as language and document type) while still being near the start. We will also show that picking different numbers here does not change our conclusions.

Finally, it is worth noting that in-context learning is of potentially special relevance to safety. In-context learning makes it harder to anticipate how a model might behave after a long context. In the longer run, concepts such as mesa-optimization or inner-alignment [14] postulate that meaningful learning or optimization could occur at test time (without changing the weights). In-context learning would be an obvious future mechanism for such hidden optimization to occur, whether or not it does so today. Thus, studying in-context learning seems valuable for the future.

(See Related Work for more on in-context learning, and Discussion for more on the connection to safety.)

Induction Heads

In our previous paper, we discovered a special kind of attention head – which we named *induction heads* – in two layer attention-only models. Induction heads are implemented by a circuit consisting of a pair of attention heads in different layers that work together to copy or complete patterns. The first attention head copies information from the previous token into each token. This makes it possible for the second attention head to attend to tokens based on what happened before them, rather than their own content. Specifically, the second head (which we call the "induction head") search for a previous place in the sequence where the present token `A` occurred and attends to the next token (call it `B`), copying it and causing the model to be more likely to output `B` as the next token. That is, the two heads working together cause the sequence `…[A][B]…[A]` to be more likely to be completed with `[B]`.

Induction heads are named by analogy to inductive reasoning. In inductive reasoning, we might infer that if `A` is followed by `B` earlier in the context, `A` is more likely to be followed by `B` again later in the same context. Induction heads crystallize that inference. They search the context for previous instances of the present token, attend to the token which would come next if the pattern repeated, and increase its probability. Induction heads attend to tokens that would be predicted by basic induction (over the context, rather than over the training data).

Notice that induction heads are implementing a simple algorithm, and are *not* memorizing a fixed table of n-gram statistics. The rule `[A][B] … [A] → [B]` applies regardless of what `A` and `B` are.[4] This means that induction heads can in some sense work *out of distribution,* as long as local statistics early in the context are representative of statistics later. This hints that they may be capable of more general and abstract behavior.

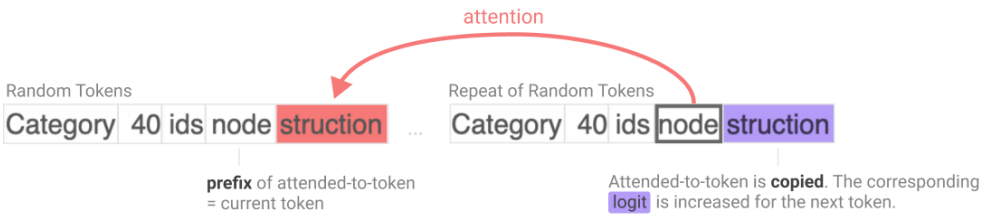
Our previous paper focused on a few explorations of induction heads, including showing that these heads occur in 2-layer attention-only models (but not 1-layer models); tracking down how they operate mechanistically as part of our mathematical decomposition of transformers; and making an eigenvalue-based test for detecting their presence. However, we were a bit vague on the exact *definition* of induction heads: it was more like we found a cluster of behaviors and mechanisms that tended to occur together, and called heads in that cluster "induction heads".

In this paper our goal is to provide evidence for something more expansive: that induction heads play a major role in *general* in-context learning (not just literal `[A][B]...[A]→[B]` copying), for *large* models and not just for small 2-layer attention only models. To do this clearly and coherently, we need a more precise definition of induction heads. Mechanistic analysis of weights and eigenvalue analysis are much more complicated in large models with MLP's, so for this paper we choose to *define* induction heads by their narrow empirical sequence copying behavior (the `[A][B]...[A]→[B]`), and then attempt to *show* that they (1) also serve a more expansive function that can be tied to in-context learning, and (2) coincide with the mechanistic picture for small models.

Formally, **we define an induction head** as one which exhibits the following two properties[5] on a repeated random sequence[6] of tokens:

- **Prefix matching:** The head attends back to previous tokens that were followed by the current and/or recent tokens.[7] That is, it attends to the token which induction would suggest comes next.
- **Copying**: The head's output increases the logit corresponding to the attended-to token.

In other words, induction heads are any heads that empirically increase the likelihood of `[B]` given `[A][B]...[A]` when shown a repeated sequence of completely random tokens. An illustration of induction heads' behavior is shown here:



Note that, as a consequence, induction heads will tend to be good at repeating sequences wholesale. For example, given "The cat sat on the mat. The cat ...", induction heads will promote the continuation "sat on the mat". This gives a first hint of how they might be connected to general in-context learning and even few-shot learning: they learn to repeat arbitrary sequences, which is a (simple) form of few-shot learning.

One of things we'll be trying to establish is that when induction heads occur in sufficiently large models and operate on sufficiently abstract representations, the very same heads that do this sequence copying *also* take on a more expanded role of *analogical sequence copying* or *in-context nearest neighbors*. By this we mean that they promote sequence completions like `[A*][B*] … [A] → [B]` where `A*` is not exactly the same token as `A` but similar in some embedding space, and also `B` is not exactly the same token as `B*`. For example, `A` and `A*` (as well as B and `B*`) might be the same word in different languages, and the induction head can then translate a sentence word by word by looking for "something like `A`", finding `A*` followed by `B*`, and then completing with "something like `B*`" (which is `B`). We are not yet able to prove mechanistically that induction heads do this in general, but in Argument 4 we show empirical examples of induction heads behaving in this way (including on translation), and in Argument 5 we point out that the known copying mechanism of induction heads in small models can be naturally adapted to function in this way.

Per-Token Loss Analysis

To better understand how models evolve during training, we analyze what we call the "per-token loss vectors." The core idea traces back to a method used by Erhan *et al.* [15], and more generally to the idea of "function spaces" in mathematics.[8]